



Programação WEB Avançada

Universidade de Vila Velha

Comprometida com a excelência no ensino, pesquisa e inovação.

✉ wanderson.santana@uvv.br

5

Estruturas básicas da aplicação

Resumo

Nesta unidade exploramos diferentes partes de uma aplicação Flask. Iremos também escrever e rodar nossa versão inicial do aplicativo *delivery*. Utilizaremos o motor de templates Jinja2, integrado nativamente ao framework. Abordaremos as boas práticas de uso de templates, passagem de contexto, filtros e macros, além da utilização da extensão Flask-DebugToolbar como ferramenta essencial de depuração visual durante o desenvolvimento. O objetivo é capacitá-los a construir interfaces dinâmicas, realizar depuração eficiente de templates e compreender o ciclo de renderização *server-side*, preparando-o para a construção de aplicações web profissionais e manuteníveis.

*“Um bom template não é apenas bonito:
é legível, reutilizável e debugável.”*
Adaptado de Armin Ronacher

Introdução

Todas as aplicações Flask devem criar uma instância da aplicação. O servidor web passa todas as requisições que recebe dos clientes para este objeto de processamento, usando um protocolo chamado **Web Server Gateway Interface (WSGI)**. Conforme discutimos na última unidade, a **instância da aplicação** é um objeto da classe Flask que, quando criado em *Factory*, apresenta a seguinte forma:

Código: `app.py`

```
1 from flask import Flask
2 def create_app():
3     app = Flask(__name__)
4
5     return app
```

O único argumento obrigatório para o construtor da classe Flask é o nome do módulo ou pacote principal da aplicação. Para a maioria das aplicações, a variável `__name__` do Python é o valor correto para este argumento.

NOTA

O argumento `__name__` que é passado para o construtor da aplicação Flask é uma fonte de confusão entre novos desenvolvedores Flask. O Flask usa este argumento para determinar a localização da aplicação, o que, por sua vez, permite localizar outros arquivos que fazem parte da aplicação, como imagens e templates.

Portanto, toda aplicação Flask gira em torno de uma instância da classe `Flask`, responsável por receber e processar requisições HTTP por meio do padrão WSGI. No entanto, apenas criar a aplicação não é suficiente: precisamos definir *como* ela responde às requisições recebidas.

Na próxima seção, veremos como associar URLs a funções Python por meio de rotas, dando comportamento à aplicação.

Rotas e Funções de Visualização

Quando um cliente (normalmente um navegador web) envia uma requisição HTTP para determinada URL, o servidor WSGI encaminha essa requisição para a instância da aplicação Flask. Cabe à aplicação decidir qual função Python será executada para processar aquela URL específica.

Essa associação entre uma URL e uma função Python é denominada **rota**. No Flask, a maneira mais direta de definir rotas é por meio do decorador `@app.route()`. E um exemplo clássico, comum em aplicações pequenas de arquivo único (não recomendado para projetos de médio ou grande porte) é o seguinte:

Código: `exemplo_simples.py`

```
1 from flask import Flask
2 app = Flask(__name__)
3
4 @app.route('/')
5 def index():
6     return '<h1>Bem-vindo!</h1>'
```

No entanto, ao adotarmos o padrão **Application Factory** (ou seja, criar a aplicação dentro da função `create_app()`), surge um problema fundamental — a instância `app` só passa a existir *após* a execução de `create_app()`. Isso implica que:

- não é possível utilizar `@app.route(...)` no momento da importação de um módulo, pois `app` ainda não foi instanciado;
- qualquer tentativa de importar módulos contendo `@app.route` antes da criação da aplicação resulta em:

```
AttributeError: 'NoneType' object has no attribute '↵
↵route'.
```

A solução oficial e recomendada pelo Flask para aplicações estruturadas é o uso de **Blueprints**.

Conforme já discutimos, um Blueprint pode ser entendido como um módulo reutilizável que permite definir rotas e funções de visualização, registrar manipuladores de erro específicos (ex.: 404, 500), adicionar filtros de contexto e processadores de template, além de associar pastas próprias de templates e static (ou compartilhar as do app principal).

O aspecto mais relevante no contexto do padrão Factory é que: **o blueprint não executa nada imediatamente**. Ele permanece em estado *dormente* (inativo) até ser explicitamente registrado na aplicação com `app.register_blueprint()`. Isso resolve o problema de inicialização tardia da instância app. Veremos isso na prática!

NOTA

Ao utilizar o padrão *Application Factory*, não podemos mais definir rotas diretamente com `@app.route`, pois a aplicação ainda não foi instanciada no momento da importação. Os **blueprints** surgem como a solução idiomática para esse problema, permitindo definir rotas de forma desacoplada e registrá-las posteriormente na aplicação.

Além de resolver a limitação técnica, essa abordagem também abre caminho para a modularização do sistema, um aspecto essencial em aplicações reais.

Estrutura recomendada com Blueprints

A organização idiomática segue estes passos:

1. criar o objeto Blueprint no escopo global do módulo (neste projeto, faremos dentro do pacote `delivery/views/main.py`);
2. definir as rotas utilizando o decorador do Blueprint (por exemplo, `@bp.route`), e nunca `@app.route`, já que estamos trabalhando com **Factory**;
3. criar uma função `init_app(app)` responsável por registrar o blueprint na aplicação — isso será feito no `__init__.py` do pacote `delivery/views`;
4. no `create_app()`, importar e invocar a função `init_app(app)` de cada blueprint.

NOTA

O objetivo desta implementação é aplicarmos os conceitos apresentados em um projeto organizado com *Application Factory* e Blueprints. O foco neste momento não é a complexidade do sistema, mas sim compreender como definir rotas de forma modular, como registrá-las na aplicação principal e como o Flask organiza internamente o fluxo de execução.

Aplicando ao nosso projeto de *delivery*, temos a seguinte implementação do nosso escopo global do módulo dedicado às rotas e funções de visualização:

Código: `delivery/views/main.py`

```
1 from flask import Blueprint, current_app
2 bp_main = Blueprint("main", __name__)
3
4 @bp_main.route('/')
5 @bp_main.route('/index')
6 def index():
7     """
8     Pagina inicial do site de delivery.
9     """
10    current_app.logger.debug("Acesso registrado a pagina inicial (/ ou /index)")
11    user = {'username': 'Wanderson'}
12    return f'''
13    <html>
14        <head><title>Home Page - Delivery UVV</title></head>
15        <body>
16            <h1>Ola, {user['username']}!</h1>
17        </body>
18    </html>'''
19 @bp_main.route('/teste')
20 def teste():
21     """
22     Rota simples para validacao do blueprint e do registro.
23     """
24    current_app.logger.info("Rota de teste /teste acessada com sucesso")
25    return "<h1 style='color: green'>Teste OK! Rota /teste funcionando.</h1>"
```

Na sequência, nosso arquivo responsável por registrar o Blueprint criado:

Código: `delivery/views/__init__.py`

```
1 """
2 Views representam a camada de apresentacao da aplicacao.
3 Responsaveis por definir rotas e retornar respostas HTTP ao cliente.
4 """
5 from delivery.views.main import bp_main
6
7 def init_app(app):
8     """
9     Registra o blueprint na aplicacao Flask e envia mensagem de inicializacao.
10    """
11    app.register_blueprint(bp_main)
12    app.logger.info("Blueprint 'site' registrado com sucesso")
```

O código `delivery/views/main.py` inicia com a importação de dois elementos fundamentais da biblioteca Flask: `Blueprint` e `current_app`. Em seguida, é criado um objeto `Blueprint` denominado `bp_main`. Esse objeto representará o conjunto de rotas e funcionalidades relacionadas na parte inicial da nossa aplicação, funcionando como um componente modular. Novamente, o parâmetro `__name__` é utilizado para que o Flask consiga localizar corretamente arquivos associados, como templates e recursos estáticos.

O código define então duas rotas distintas, `/` e `/index`, ambas associadas à função `index`. Essa prática é comum em aplicações web, pois permite que diferentes URLs levem ao mesmo conteúdo, melhorando a experiência do usuário e a flexibilidade da navegação.

Dentro da função `index`, observa-se o uso do sistema de logs da aplicação por meio do método `debug`. Esse tipo de registro é especialmente útil durante o desenvolvimento, pois permite acompanhar o fluxo de execução e identificar possíveis problemas.

Um ponto importante do código é a criação de um objeto simulado representando um usuário:

```
1 user = \{'username': 'Wanderson'\}
```

A utilização de dados simulados (também conhecidos como *mocks*) é uma técnica amplamente empregada no desenvolvimento de software. Ela permite que o desenvolvedor

foque em partes específicas da aplicação — neste caso, a construção da rota e da resposta HTML — sem depender de outras camadas ainda não implementadas, como o banco de dados ou sistemas de autenticação. Dessa forma, é possível avançar no desenvolvimento de forma incremental e mais controlada.

Na sequência, a função retorna uma string contendo código HTML. O uso de *f-strings* em Python permite a inserção dinâmica de valores dentro da string, como o nome do usuário, tornando a resposta mais personalizada.

Posteriormente, o código define uma nova rota, acessível por meio do caminho `/teste`. Essa rota está associada à função `teste`, cuja finalidade é validar se o *Blueprint* foi corretamente registrado na aplicação.

Dentro dessa função, utiliza-se o método `info` do sistema de logs, que registra eventos relevantes em um nível mais apropriado para ambientes de produção. Por fim, a função retorna uma mensagem HTML simples, estilizada com CSS inline, indicando que a rota está funcionando corretamente.

Por fim, o factory principal configura o logging básico (para desenvolvimento) e registra o blueprint:

Código: `app.py`

```
1 import logging
2 from flask import Flask
3
4 def create_app():
5     app = Flask(__name__)
6     # Configuracao recomendada para desenvolvimento:
7     # Mostra mensagens DEBUG e INFO no console
8     if app.debug:
9         app.logger.setLevel(logging.DEBUG)
10
11     # =====
12     # REGISTRO BLUEPRINT
13     # =====
14
15     from delivery.views import init_app as init_site
16     init_site(app)
17     return app
```


Com essa estrutura, conseguimos organizar nossa aplicação de forma modular, utilizando **Blueprints** para definir rotas e o padrão *Application Factory* para controlar a criação da aplicação. Essa separação permite maior flexibilidade, facilita testes e prepara o projeto para crescer de maneira organizada.

Nos próximos passos, evoluiremos essa base adicionando novas camadas e responsabilidades ao sistema.

Aviso

Evolução da arquitetura

Neste primeiro momento, estamos organizando o projeto apenas com a camada de *views*, com o objetivo de focar no funcionamento de rotas e blueprints.

Em aplicações reais, é comum adotar uma arquitetura em camadas, separando responsabilidades como:

- **models**: definição das estruturas de dados;
- **repositories**: acesso a banco de dados;
- **services**: regras de negócio;
- **views**: interface HTTP (rotas e respostas).

Essa evolução será realizada gradualmente ao longo das próximas unidades, por meio de refatorações sucessivas do código.

Por que usar `current_app.logger` nos blueprints?

Dentro de um blueprint (que é definido fora do contexto da aplicação ativa), não temos acesso direto à instância `app`. A solução idiomática do Flask é utilizar o proxy `current_app`, que aponta para a aplicação ativa no contexto de requisição ou durante a inicialização. Assim:

- `app.logger` → usado dentro de `create_app()` ou em funções que recebem `app` como argumento;
- `current_app.logger` → usado em rotas, filtros, manipuladores de erro e qualquer código executado dentro do contexto da aplicação.

O uso de `current_app` permite acessar a aplicação ativa mesmo em contextos onde a instância `app` não está diretamente disponível, como dentro de blueprints. Esse mecanismo é fundamental para manter o desacoplamento entre os módulos da aplicação, preservando a organização proposta pelo padrão Factory.

Vantagens práticas dessa estrutura

- **Modularidade forte:** cada pacote (ex.: `site (main)`, `admin`, `api`) tem suas próprias rotas, templates, arquivos estáticos e lógica;
- **Testabilidade:** blueprints podem ser testados de forma isolada (sem precisar levantar a aplicação inteira);
- **Prefixos de URL:** possibilidade de registrar com `url_prefix='/admin'` ou `'/api/v1'`, evitando conflitos;
- **Evita importações circulares:** a separação clara entre inicialização e definição de rotas elimina erros comuns;
- **Escalabilidade:** facilita adicionar novos módulos (autenticação, painel administrativo, API REST, etc.) sem alterar o núcleo da aplicação;
- **Logging estruturado:** mensagens claras de inicialização e depuração, com níveis apropriados (DEBUG em dev, INFO/WARNING em produção).

Com essa arquitetura, o projeto ganha uma base sólida, idiomática e alinhada às melhores práticas atuais do ecossistema Flask. No entanto, ainda estamos gerando HTML diretamente nas funções de visualização, o que não é ideal para aplicações reais.

Caso precise revisar ou aprofundar conceitos de HTML (tags, estrutura de página, atributos, formulários, semântica moderna, elementos acessíveis etc.), recomendo fortemente consultar a documentação oficial em português do MDN Web Docs.

Para uma abordagem prática e interativa, complemente com o curso gratuito de **Responsive Web Design** da freeCodeCamp.

O Flask oferece suporte robusto para geração de páginas HTML dinâmicas por meio de **templates** (utilizando o mecanismo Jinja2). Esse recurso é essencial para separar a lógica da aplicação da apresentação, evitando a construção de HTML diretamente no código Python.

Na próxima seção, introduziremos o uso de **templates**, separando a lógica da aplicação da camada de apresentação e dando mais um passo em direção a uma arquitetura profissional.

Templates

A chave para construir aplicações web fáceis de manter e escalar está na **separação clara de responsabilidades**. Nos exemplos iniciais do nosso projeto de delivery, as funções de visualização (views) retornam HTML diretamente — uma abordagem simples, porém inadequada para aplicações reais.

É crucial que compreendamos aqui que toda função de visualização no Flask acaba lidando com responsabilidades que parecem se misturar. Entretanto, elas precisam ser tratadas de maneira separadas. Ou seja, não devemos misturar:

- **Lógica da aplicação:** preparação de dados, chamadas a serviços, validações e regras do sistema, com;
- **Lógica de apresentação** (*presentation logic*): que é a geração do HTML exibido ao usuário.

Quando essas responsabilidades são combinadas, o código se torna difícil de manter, testar e evoluir. Por exemplo:

```
1 return '<h1>Ola, ' + user['username'] + '!</h1><p>Seu pedido #' + str(pedido_id) + '  
    foi confirmado.</p>'
```

Esse tipo de construção rapidamente se torna ilegível e propenso a erros.

Logo, a solução é mover a **lógica de apresentação** para arquivos separados chamados **templates**. Um template é um arquivo (geralmente '.html') que contém a estrutura da página com **espaços reservados** (*placeholders*) para dados dinâmicos.

O Flask utiliza o mecanismo de templates **Jinja2**, responsável por realizar a **renderização** — ou seja, substituir variáveis por valores reais e gerar o HTML final. Essa abordagem segue o princípio fundamental de engenharia de software conhecido como *separation of concerns* (separação de responsabilidades).

NOTA

Ao introduzir templates, eliminamos a necessidade de construir HTML manualmente nas funções de visualização, tornando o código mais limpo, legível e sustentável.

Além disso, passamos a separar claramente a camada de controle (views) e a camada de apresentação (templates).

Nesta seção e nas seguintes, abordaremos:

- funcionamento da renderização e contexto automático do Flask;
- uso de variáveis, filtros, condicionais, loops, macros e herança de templates;
- depuração visual com Flask-DebugToolbar;
- integração da toolbar no Application Factory;
- boas práticas de segurança (autoescape, prevenção de XSS) e performance na camada de apresentação.

Renderização de Templates com Jinja2

Na forma mais simples, um template Jinja2 é um arquivo HTML com marcações especiais delimitadas por:

- `{{ ... }}` → expressões (variáveis, filtros);

- {% ... %} → estruturas de controle (if, for, block, extends etc.);
- {# ... #} → comentários.

O Flask procura templates por padrão na pasta `templates/` (relativa à raiz da aplicação ou ao blueprint). Na sequência dos exemplos a seguir, daremos continuidade ao projeto de delivery, enfatizando as vantagens de utilização do Jinja2.

Crie uma pasta `templates/main/` (associada ao blueprint) e adicione o seguinte código:

Código: templates/main/index.html

```

1 <!DOCTYPE html>
2 <html lang="pt-br">
3 <head>
4     <meta charset="UTF-8">
5     <title>Home Page - Delivery UVV</title>
6 </head>
7 <body>
8     <h1>Ola, {{ user.username }}!</h1>
9     <p>Bem-vindo ao nosso sistema de delivery.</p>
10 </body>
11 </html>

```

Note que utilizamos `{{ user.username }}` em vez de concatenar strings no código Python. Isso melhora a legibilidade e separa claramente responsabilidades.

Agora, ajuste a rota para renderizar o template:

Código: delivery/views/main.py (trecho atualizado)

```

1 from flask import Blueprint, render_template, current_app
2 bp_main = Blueprint("main", __name__)
3
4 @bp_main.route('/')
5 @bp_main.route('/index')
6 def index():
7     current_app.logger.debug("Renderizando template index.html")
8     user = {'username': 'wanderson'}
9     return render_template('main/index.html', user=user)

```

A função `render_template()`:

- localiza o template pelo caminho relativo (ex.: `'main/index.html'`);
- injeta variáveis no contexto do template;
- executa o mecanismo Jinja2;
- retorna um objeto `Response` com conteúdo HTML.

Os argumentos nomeados são inseridos no contexto do template. O nome à esquerda é o identificador usado no HTML:

```
1 return render_template('main/index.html', user=user) # {{ user.username }}
```

Segurança na renderização (autoescape)

Sempre que exibimos dados dinâmicos em HTML, existe o risco de que esses dados contenham código malicioso inserido intencionalmente por um atacante ou acidentalmente por um usuário. Um exemplo clássico é o seguinte conteúdo:

```
1 <script>alert('XSS')</script>
```

Se esse conteúdo for inserido diretamente na página sem qualquer proteção, o navegador o interpretará como código JavaScript válido e o executará — caracterizando um ataque conhecido como **Cross-Site Scripting (XSS)**.

Existem basicamente três tipos principais de XSS:

- **Reflected XSS**: o payload¹ malicioso é refletido imediatamente na resposta (ex.: via parâmetro de query string);

¹**Payload**, neste contexto, refere-se ao conteúdo (geralmente uma string) que está sendo injetado ou transmitido para ser processado e exibido na página web. No exemplo de ataque XSS, o *payload* é o trecho de código (como `<script>alert('XSS')</script>`) que o atacante tenta fazer com que o navegador execute. Em demonstrações de segurança, usamos um *payload* controlado e inofensivo para simular o comportamento de um ataque real, sem causar danos. O termo vem do inglês e significa literalmente “carga útil” ou “carga transportada”, indicando a parte “perigosa” ou “importante” que está sendo carregada dentro de uma requisição ou resposta HTTP.

- **Stored XSS:** o payload é armazenado no servidor (ex.: comentário em banco de dados) e servido a todos os visitantes;
- **DOM-based XSS:** a vulnerabilidade ocorre no lado do cliente, manipulando o DOM via JavaScript.

No contexto do Flask + Jinja2, o tipo mais comum em aplicações mal configuradas é o **Reflected** ou **Stored** via templates.

Para evitar isso, o Flask (por meio do Jinja2) ativa **automaticamente** o mecanismo de **autoescape** em templates com extensão `.html`, `.htm`, `.xml` etc. Esse mecanismo converte caracteres especiais em suas entidades HTML seguras antes de enviar a resposta ao navegador:

- Entrada original: `<script>alert('XSS')</script>`
- Saída segura (autoescape ativado): `<script>alert('XSS')</script>`

Ou seja, o conteúdo é exibido como texto puro, e o navegador **não** o executa como código.

Demonstração prática controlada

Para ilustrar o risco e a proteção, criamos uma rota de demonstração temporária `/xss-demo` (presente apenas para fins educacionais). Nela, simulamos um payload malicioso:

Código: trecho de `delivery/views/main.py`

```

1 @bp_main.route('/xss-demo')
2 def xss_demo():
3     current_app.logger.warning(
4         "DEMO DIDATICA DE XSS - NAO USE ISSO COM DADOS DO USUARIO EM PRODUCAO!"
5     )
6     payload = "<script>alert('XSS - demonstrando o risco do |safe')</script>"
7     user = {'username': payload}
8     return render_template('main/index.html', user=user, demo_mode=True)

```

No template associado (`templates/main/index.html`), adicionamos um bloco condicional didático:

Código: templates/main/index.html (trecho)

```

1  {% if demo_mode %}
2      <hr>
3      <h2 style="color: red;">DEMONSTRACAO DIDATICA - XSS com | safe (apenas para
        aula!)</h2>
4      <p>Payload original: <code>{{ "<script>alert('XSS')</script>" }}</code></p>
5
6      <!-- Aqui acontece o XSS intencional para fins educativos -->
7      <p>Resultado com |safe: {{ user.username | safe }}</p>
8
9      <p style="color: orange; font-weight: bold;">
10         Se voce viu uma caixa de alerta aparecer, significa que o filtro "safe"
11         desativou a protecao automatica (comportamento perigoso com dados nao
12         confiaveis).
13     </p>
14 {% endif %}

```

Ao acessar <http://localhost:5000/xss-demo>, você verá: - Na linha sem safe|: o código aparece como texto puro (protegido); - Na linha com safe|: o script é executado e o alerta aparece.

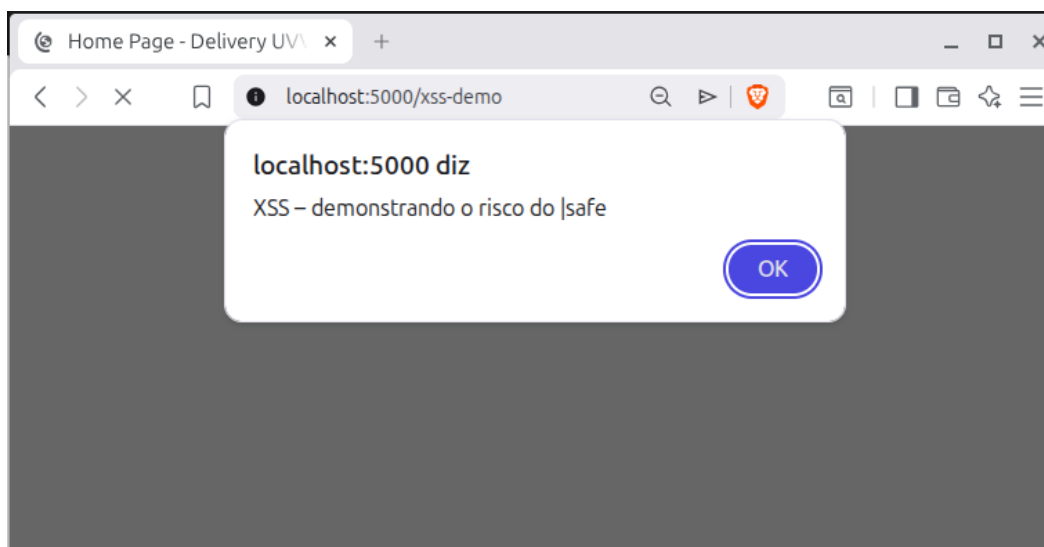


Figura 5.1: Resultado da demonstração: alerta executado após uso intencional do filtro |safe (apenas para fins educacionais).



Figura 5.2: Comportamento padrão e seguro: código exibido como texto puro graças ao autoescape.

Regras de ouro para evitar XSS em aplicações Flask

- **Nunca** desative o autoescape globalmente (ex.: `app.jinja_env.autoescape = False`);
- **Não use** o filtro `safe|` ou a classe `Markup` em conteúdo que venha (direta ou indiretamente) de usuários, banco de dados, APIs externas ou qualquer fonte não confiável;
- Use `safe|` **somente** quando você tem 100% de controle sobre o conteúdo (ex.: HTML gerado pelo próprio código da aplicação, como ícones SVG internos ou blocos de formatação Markdown confiáveis processados por bibliotecas seguras como `bleach` ou `markdown-it-py`);
- Em aplicações industriais, adote camadas adicionais de defesa:
 - Sanitização explícita de entrada com bibliotecas como `bleach` antes de armazenar no banco;
 - Validação rigorosa de todos os dados de entrada;
 - Cabeçalho **Content-Security-Policy (CSP)** para restringir fontes de scripts;
 - Uso de `Flask-Talisman` para configurar cabeçalhos de segurança automaticamente;

- Ferramentas de análise estática (ex.: Bandit, Semgrep) para detectar usos perigosos de `safe|` ou Markup.

Importante: Em produção, todo dado vindo do usuário (incluindo campos de formulário, comentários, nomes, bios etc.) deve ser tratado como potencialmente perigoso. O autoescape do Jinja2 é uma excelente primeira linha de defesa, mas não substitui uma abordagem de defesa em profundidade.

NOTA

A função `render_template()` atua como a ponte segura entre a lógica da aplicação e a interface do usuário. Ao respeitarmos o autoescape e evitarmos o uso indiscriminado de `safe|`, geramos páginas dinâmicas de forma organizada e protegida contra um dos ataques mais comuns na web.

Na próxima subseção, exploramos como manipular variáveis, aplicar filtros, condicionais e loops dentro dos templates Jinja2, ampliando expressivamente o poder de personalização da camada de apresentação — sempre mantendo a segurança em mente.

Variáveis

A construção `{{ user.username }}` refere-se a uma variável disponível no contexto do template. Essas variáveis são fornecidas pela função `render_template()`, que injeta dados da aplicação no ambiente de renderização do Jinja2.

Em outras palavras, o fluxo é:

Python (view) → `render_template` → Jinja2 → HTML final

O Jinja2 oferece diferentes formas de acessar dados:

- Acesso por ponto: `{{ user.username }}` (atributo ou chave de dicionário);
- Acesso por subscrito: `{{ user['username'] }}` ou `{{ minha_lista[0] }}`;
- Índices variáveis: `{{ minha_lista[indice] }}`;

- Chamadas de método: `{{ objeto.metodo() }}`.

Observação importante: embora chamadas de método sejam permitidas, deve-se evitar lógica complexa nos templates. Templates devem permanecer focados na apresentação, não na regra de negócio.

Exemplos comuns no contexto do delivery:

```
{{ user.username }}           {# Wanderson #}
{{ pedido.itens|length }}      {# número de itens no pedido #}
{{ data_atual | strftime('%d/%m/%Y') }} {# se data_atual for datetime #}
{{ preco | round(2) }}        {# arredonda para 2 casas decimais #}
```

Filtros

Variáveis podem ser transformadas usando **filtros**, aplicados com o operador `|`.

Filtros comuns do Jinja2 incluem:

Filtro	Descrição
upper	Converte para maiúsculas
lower	Converte para minúsculas
capitalize	Primeira letra maiúscula, resto minúsculo
title	Cada palavra com primeira letra maiúscula
default('valor')	Valor padrão se variável indefinida ou falsa
safe	Marca como HTML seguro (ignora autoescape)
escape ou e	Força escape HTML
length	Tamanho de lista/string
join(', ')	Une lista com separador
striptags	Remove tags HTML

Tabela 5.1: Filtros Jinja2 mais usados (seleção).

Exemplo:

```
1 Ola, {{ user.username | upper }}!
2 Voce tem {{ itens_carrinho | default([]) | length }} itens no carrinho.
```

Variáveis indefinidas e valores padrão

Nem sempre uma variável estará disponível no template. Isso pode ocorrer quando:

- a variável não foi enviada pelo `render_template`;
- o valor é `None`;
- o valor é considerado “falso” (ex.: lista vazia, string vazia, zero).

Para evitar erros ou comportamentos inesperados, utilizamos o filtro `default`:

```
1 {{ usuario.nome | default('Visitante') }}
```

Isso garante que o template continue funcionando mesmo quando os dados não estiverem completos.

Podemos concluir então que as variáveis em templates permitem conectar os dados da aplicação à interface do usuário de forma clara e organizada. Com o uso de filtros, conseguimos transformar e proteger esses dados diretamente na camada de apresentação, mantendo o código Python limpo e focado na lógica da aplicação.

Na próxima subseção, veremos como controlar o fluxo de renderização utilizando estruturas condicionais e loops, permitindo criar páginas dinâmicas mais complexas.

Estruturas de Controle

O Jinja2 oferece estruturas de controle poderosas para alterar o fluxo de renderização do template, semelhantes a condicionais e loops no Python. Esses recursos permitem que o conteúdo exibido ao usuário seja dinâmico, adaptando-se aos dados da aplicação.

A seguir, apresentamos as estruturas mais utilizadas, com exemplos adaptados ao nosso projeto de delivery.

Condicionais (if / else / elif)

Instruções condicionais permitem exibir conteúdo diferente com base no valor de variáveis ou expressões, de forma semelhante ao “se-então-senão” do Python.

Exemplo prático — saudação personalizada dependendo se o usuário está logado:

Código: templates/main/index.html (trecho)

```
1 {% if user %}  
2     <h1>Ola, {{ user.username | capitalize }}!</h1>  
3     <p>Bem-vindo de volta ao Delivery UVV.</p>  
4 {% else %}  
5     <h1>Ola, visitante!</h1>  
6     <p>Faca login ou crie uma conta para pedir agora.</p>  
7 {% endif %}
```

Nesse contexto, a variável `user` é avaliada como verdadeira se existir e tiver conteúdo, e como falsa caso seja `None`, vazia ou não definida.

Podemos estender a lógica com `elif` para tratar múltiplas condições:

Código: templates/main/base.html (trecho de navegação)

```
1 {% if user and user.is_admin %}  
2     <a href="/admin">Painel Admin</a>  
3 {% elif user %}  
4     <a href="/perfil">Meu Perfil</a>  
5 {% else %}  
6     <a href="/login">Entrar</a>  
7 {% endif %}
```

Observação: ao acessar atributos como `user.is_admin`, é importante garantir que `user` não seja nulo, evitando erros de execução.

Essas estruturas são fundamentais para construir interfaces dinâmicas: menus personalizados, mensagens de erro, notificações e conteúdos condicionais são exemplos comuns.

NOTA

As estruturas condicionais permitem que o template tome decisões com base nos dados recebidos, tornando a interface mais inteligente e adaptável ao contexto do usuário.

Com isso, avançamos da simples exibição de dados para a construção de interfaces realmente dinâmicas.

Na sequência, veremos como repetir blocos de conteúdo utilizando loops, permitindo exibir listas de dados como produtos, pedidos e itens do carrinho.

Agora que sabemos como tomar decisões no template, precisamos de uma forma eficiente de repetir estruturas de conteúdo. Por exemplo, listar produtos disponíveis, exibir itens no carrinho e mostrar pedidos recentes.

Para isso, o Jinja2 oferece o loop `for`, que veremos a seguir.

Loops (for)

Loops são ideais para renderizar listas de dados, como itens do carrinho ou pedidos recentes. No Jinja2, utilizamos a estrutura `for` para iterar sobre coleções de forma semelhante ao Python.

Código: templates/main/carrinho.html

```
1 <!DOCTYPE html>
2 <html lang="pt-br">
3 <head>
4     <meta charset="UTF-8">
5     <meta name="viewport" content="width=device-width, initial-scale=1.0">
6     <title>{{ titulo | default("Meu Carrinho") }}</title>
7     <style>
8         body {
9             font-family: Arial, sans-serif;
10            margin: 20px;
11            background: #f8f9fa;
12        }
```

```

13     h1 { color: #333; text-align: center; }
14     .itens-carrinho {
15         list-style: none;
16         padding: 0;
17         max-width: 800px;
18         margin: 0 auto 20px;
19     }
20     .item-carrinho {
21         display: flex;
22         justify-content: space-between;
23         align-items: center;
24         padding: 16px;
25         border-bottom: 1px solid #ddd;
26         background: white;
27         margin-bottom: 8px;
28         border-radius: 6px;
29         box-shadow: 0 1px 3px rgba(0,0,0,0.1);
30     }
31     .info-produto { flex: 3; }
32     .info-produto h3 { margin: 0 0 4px; color: #2c3e50; }
33     .info-produto p { margin: 0; color: #7f8c8d; }
34     .preco { flex: 1; text-align: right; font-weight: bold; color: #27ae60; }
35     .vazio {
36         color: #888;
37         font-style: italic;
38         padding: 40px;
39         text-align: center;
40         background: #f9f9f9;
41         border: 2px dashed #ccc;
42         border-radius: 8px;
43         max-width: 600px;
44         margin: 0 auto;
45     }
46     .total {
47         text-align: right;
48         font-size: 1.3em;
49         margin: 20px 0;
50         max-width: 800px;
51         margin-left: auto;

```

```

52     }
53     .voltar { text-align: center; margin-top: 30px; }
54 </style>
55 </head>
56 <body>
57     <h1>Meu Carrinho</h1>
58
59     <ul class="itens-carrinho">
60     {% for item in carrinho.itens %}
61         <li class="item-carrinho">
62             <div class="info-produto">
63                 <h3>{{ item.nome }}</h3>
64                 <p>Quantidade: {{ item.quantidade }} x R$ {{ item.preco | round(2)
65                     }}</p>
66             </div>
67             <div class="preco">
68                 R$ {{ (item.preco * item.quantidade) | round(2) }}
69             </div>
70         </li>
71     {% else %}
72         <li class="vazio">0 carrinho esta vazio.<br>Adicione itens para continuar!</li>
73     {% endfor %}
74 </ul>
75
76     {% if carrinho.itens %}
77         <div class="total">
78             <strong>Total estimado: R$ {{ total | round(2) }}</strong>
79         </div>
80     {% endif %}
81
82     <div class="voltar">
83         <p><a href="/">Voltar para a pagina inicial</a></p>
84     </div>
85 </body>
</html>

```

O bloco {% else %} é opcional e será executado automaticamente caso a lista esteja

vazia. Isso elimina a necessidade de criar condicionais extras apenas para verificar se há itens.

Além disso, o Jinja2 fornece uma variável especial chamada `loop`, disponível dentro do bloco `for`, com informações úteis sobre a iteração:

- `loop.index` → posição atual (começa em 1);
- `loop.first` → verdadeiro no primeiro elemento;
- `loop.last` → verdadeiro no último elemento.

Exemplo:

```
1 {% for item in carrinho.itens %}
2     {{ loop.index }} - {{ item.nome }}
3 {% endfor %}
```

Agora, veja a rota correspondente no Flask:

Código: `delivery/views/main.py` (atualização)

```
1 @bp_main.route('/carrinho')
2 def carrinho():
3     carrinho = {
4         'itens': [
5             {'id': 1, 'nome': 'Pizza Margherita', 'preco': 49.95, 'quantidade': 1},
6             {'id': 2, 'nome': 'Refrigerante 2L', 'preco': 8.52, 'quantidade': 2},
7             {'id': 3, 'nome': 'Borda Recheada', 'preco': 12.358, 'quantidade': 1},
8         ]
9         # Para testar vazio: 'itens': []
10    }
11    total = sum(item['preco'] * item['quantidade'] for item in carrinho['itens'])
12    current_app.logger.debug(f"Renderizando carrinho com {len(carrinho['itens'])}
13                             itens | Total: R$ {total:.2f}")
14    return render_template(
15        'main/carrinho.html',
16        carrinho=carrinho,
17        total=total,
18        titulo="Meu Carrinho - Delivery UVV")
```

NOTA

Os loops permitem iterar sobre coleções de dados e gerar conteúdo repetido de forma eficiente, sendo essenciais para exibir listas dinâmicas como produtos, pedidos e itens de carrinho.

O uso combinado de `for`, `else` e da variável `loop` oferece grande flexibilidade na construção de interfaces dinâmicas.

Na próxima subseção, veremos como reutilizar trechos de templates por meio de **macros**, evitando repetição de código e melhorando a organização da camada de apresentação.

Macros reutilizáveis

Macros no Jinja2 funcionam de forma semelhante a funções: permitem definir blocos reutilizáveis de template, evitando repetição de código e facilitando a manutenção. São especialmente úteis para componentes repetitivos, como itens de lista, botões, formulários ou cartões de produto.

A seguir, definimos uma macro para renderizar um item do carrinho:

Código: templates/main/macros.html

```
1 {% macro item_carrinho(item) %}
2 <li class="item-carrinho">
3     <div class="info-produto">
4         <h3>{{ item.nome }}</h3>
5         <p>Quantidade: {{ item.quantidade }} x R$ {{ item.preco | round(2)
6             }}</p>
7     </div>
8     <div class="preco">
9         R$ {{ (item.preco * item.quantidade) | round(2) }}
10    </div>
11 </li>
12 {% endmacro %}
```

Agora, utilizamos essa macro dentro do template do carrinho:

Código: templates/main/carrinho.html (atualizado)

```
1 <!DOCTYPE html>
2 <html lang="pt-br">
3 <head>
4     <meta charset="UTF-8">
5     <meta name="viewport" content="width=device-width, initial-scale=1.0">
6     <title>{{ titulo | default("Meu Carrinho") }}</title>
7     <style>
8         body {
9             font-family: Arial, sans-serif;
10            margin: 20px;
11            background: #f8f9fa;
12        }
13        h1 { color: #333; text-align: center; }
14        .itens-carrinho {
15            list-style: none;
16            padding: 0;
17            max-width: 800px;
18            margin: 0 auto 20px;
19        }
20        .item-carrinho {
21            display: flex;
22            justify-content: space-between;
23            align-items: center;
24            padding: 16px;
25            border-bottom: 1px solid #ddd;
26            background: white;
27            margin-bottom: 8px;
28            border-radius: 6px;
29            box-shadow: 0 1px 3px rgba(0,0,0,0.1);
30        }
31        .info-produto { flex: 3; }
32        .info-produto h3 { margin: 0 0 4px; color: #2c3e50; }
33        .info-produto p { margin: 0; color: #7f8c8d; }
34        .preco { flex: 1; text-align: right; font-weight: bold; color: #27ae60; }
35        .vazio {
36            color: #888;
37            font-style: italic;
```

```
38         padding: 40px;
39         text-align: center;
40         background: #f9f9f9;
41         border: 2px dashed #ccc;
42         border-radius: 8px;
43         max-width: 600px;
44         margin: 0 auto;
45     }
46     .total {
47         text-align: right;
48         font-size: 1.3em;
49         margin: 20px 0;
50         max-width: 800px;
51         margin-left: auto;
52     }
53     .voltar { text-align: center; margin-top: 30px; }
54 </style>
55 </head>
56 <body>
57     <h1>Meu Carrinho</h1>
58
59     {% import 'main/macros.html' as macros %}
60
61     <ul class="itens-carrinho">
62         {% for item in carrinho.itens %}
63             {{ macros.item_carrinho(item) }}
64         {% else %}
65             <li class="vazio">0 carrinho esta vazio.<br>Adicione itens para continuar!</li>
66         {% endfor %}
67     </ul>
68
69     {% if carrinho.itens %}
70         <div class="total">
71             <strong>Total estimado: R$ {{ total | round(2) }}</strong>
72         </div>
73     {% endif %}
74
75     <div class="voltar">
76         <p><a href="/">Voltar para a pagina inicial</a></p>
```

```
77     </div>
78 </body>
79 </html>
```

A diretiva:

```
1 {% import 'main/macros.html' as macros %}
```

carrega todas as macros do arquivo e as disponibiliza com o prefixo `macros`.

Também é possível importar apenas funções específicas:

```
1 {% from 'main/macros.html' import item_carrinho %}
2 {{ item_carrinho(item) }}
```

Aviso

Macros devem ser usadas para lógica de apresentação reutilizável.
Evite incluir regras de negócio dentro delas.

Podemos concluir que macros permitem encapsular trechos reutilizáveis de interface, reduzindo duplicação de código e facilitando a manutenção dos templates. Esse conceito aproxima o desenvolvimento com Jinja2 de abordagens modernas baseadas em componentes, amplamente utilizadas em *frameworks frontend*.

Na próxima subseção, veremos como estruturar templates de forma hierárquica utilizando herança (*template inheritance*), permitindo definir layouts base reutilizáveis.

Herança de templates (extends e blocks)

A herança de templates é o recurso mais poderoso do Jinja2 para manter um layout consistente em todo o aplicativo, evitando repetição de código HTML comum (como header, footer, navegação e inclusão de arquivos CSS/JS).

O funcionamento é semelhante à herança de classes no Python: um template base define a estrutura geral da página, e os templates filhos herdam essa estrutura, sobrescrevendo apenas as partes necessárias.

Modelo mental:

- **base.html** → define o layout global da aplicação;
- **templates filhos** → preenchem apenas o conteúdo específico de cada página.

Com base nesse modelo mental, vamos criar o template base com os elementos compartilhados, conforme exemplo a seguir.

Código: templates/main/base.html

```
1 <!DOCTYPE html>
2 <html lang="pt-br">
3 <head>
4     {% block head %}
5         <meta charset="UTF-8">
6         <meta name="viewport" content="width=device-width, initial-scale=1.0">
7         <title>{% block title %}Delivery UVV{% endblock %}</title>
8         {% block css %}{% endblock %}
9     {% endblock %}
10 </head>
11 <body>
12     <header>
13         <nav>
14             {% block navbar %}
15                 <ul>
16                     <li><a href="/">Inicio</a></li>
17                     <li><a href="/carrinho">Carrinho</a></li>
18                 </ul>
19             {% endblock %}
20         </nav>
21     </header>
22
23     <main>
24         {% block content %}
25         <!-- Conteudo principal sera definido pelos templates filhos -->
26         {% endblock %}
27     </main>
28
29     <footer>
```

```

30     {% block footer %}
31     <p>&copy; 2026 UVV - Vila Velha, ES | Todos os direitos reservados.</p>
32     {% endblock %}
33 </footer>
34 </body>
35 </html>

```

Agora, o template da página inicial herda esse layout:

Código: templates/main/index.html

```

1  {% extends "main/base.html" %}
2
3  {% block title %}Home{% endblock %}
4
5  {% block head %}
6      {{ super() }}
7      <style>.welcome { color: blue; text-align: center; margin: 40px 0; }</style>
8  {% endblock %}
9
10 {% block content %}
11     {% if user %}
12         <h1 class="welcome">Ola, {{ user.username | capitalize }}!</h1>
13         <p>Bem-vindo de volta ao Delivery UVV.</p>
14     {% else %}
15         <h1>Ola, visitante!</h1>
16         <p>Faca login ou crie uma conta para pedir agora.</p>
17     {% endif %}
18
19     {% if demo_mode %}
20         <hr>
21         <h2 style="color: red;">DEMONSTRACAO DIDATICA x XSS com |safe (apenas para
22             aula!)</h2>
23         <p>Payload original: <code>{{ "<script>alert('XSS')</script>" }}</code></p>
24
25         <!-- Aqui acontece o XSS intencional para fins educativos -->
26         <p>Resultado com |safe: {{ user.username | safe }}</p>
27
28         <p style="color: orange; font-weight: bold;">

```

```
28         Se voce viu uma caixa de alerta aparecer, significa que o filtro "safe"
29         desativou a protecao automatica (comportamento perigoso com dados nao
           confiaveis).
30     </p>
31     {% endif %}
32 {% endblock %}
```

Pontos importantes:

- A diretiva `{% extends "main/base.html" %}` deve ser a **primeira linha** do template;
- `{{ super() }}` permite reutilizar o conteúdo do bloco pai e adicionar extensões;
- Blocos não redefinidos no template filho mantêm o comportamento do template base;
- O uso de um layout base evita duplicação de código e facilita alterações globais;
- Essa abordagem segue diretamente o princípio DRY (Don't Repeat Yourself).

NOTA

A herança de templates permite separar claramente a estrutura global da aplicação do conteúdo específico de cada página.

Essa técnica é essencial para manter consistência visual, reduzir duplicação de código e facilitar a evolução do sistema.

Na prática, todas as páginas do sistema (index, carrinho, produtos, perfil etc.) devem estender um template base.

Veja a seguir, como fica a nossa página do `carrinho.html`

Código: `templates/main/carrinho.html` (com herança)

```
1 {% extends "main/base.html" %}
2
3 {% block css %}
4     <link rel="stylesheet" href="/static/css/style.css">
5 {% endblock %}
```



```

6
7
8 {% block content %}
9     <h1>Meu Carrinho</h1>
10
11     {% import 'main/macros.html' as macros %}
12
13     <ul class="itens-carrinho">
14         {% for item in carrinho.itens %}
15             {{ macros.item_carrinho(item) }}
16         {% else %}
17             <li class="vazio">O carrinho esta vazio.<br>Adicione itens para continuar!</li>
18         {% endfor %}
19     </ul>
20
21     {% if carrinho.itens %}
22         <div class="total">
23             <strong>Total estimado: R$ {{ total | round(2) }}</strong>
24         </div>
25     {% endif %}
26 {% endblock %}

```

Na próxima subseção, veremos como dividir ainda mais os templates utilizando inclusões parciais (*include*), permitindo reutilizar pequenos trechos como menus, rodapés e componentes específicos.

Inclusão de parciais (include)

Além da herança de templates, o Jinja2 permite reutilizar trechos menores de HTML por meio da diretiva `{% include %}`.

Esse recurso é ideal para componentes independentes e reutilizáveis, como:

- barras de navegação (navbar);
- rodapés (footer);

- banners promocionais;
- formulários de busca;
- mensagens reutilizáveis.

Diferente da herança, que define a estrutura geral da página, o `include` simplesmente insere o conteúdo de outro template no ponto onde é chamado.

Exemplo prático

Vamos criar alguns templates parciais dentro da pasta `templates/main/partials/`.

Código: `templates/main/partials/navbar.html`

```
1 <a href="/">Inicio</a>
2 <a href="/carrinho">Carrinho</a>
```

Footer:

Código: `templates/main/partials/footer.html`

```
1 <p>&copy; 2026 Delivery UVV | Todos os direitos reservados</p>
```

Agora, atualizamos o `base.html` para incluir esses componentes:

Código: `templates/main/base.html` (trecho atualizado)

```
1 <!DOCTYPE html>
2 <html lang="pt-br">
3 <head>
4     {% block head %}
5         <meta charset="UTF-8">
6         <meta name="viewport" content="width=device-width, initial-scale=1.0">
7         <title>{% block title %}Delivery UVV{% endblock %}</title>
8         {% block css %}{% endblock %}
9     {% endblock %}
10 </head>
11 <body>
12     <header>
```

```
13     <nav>
14         {% block navbar %}
15             {% include 'main/partials/navbar.html' %}
16         {% endblock %}
17     </nav>
18 </header>
19
20 <main>
21     {% block content %}
22     <!-- Conteudo principal sera definido pelos templates filhos -->
23     {% endblock %}
24 </main>
25
26 <footer>
27     {% block footer %}
28         {% include 'main/partials/footer.html' %}
29     {% endblock %}
30 </footer>
31 </body>
32 </html>
```

Importante: o template incluído recebe automaticamente todas as variáveis disponíveis no contexto atual (como user, carrinho etc.).

Comparação: include vs macros vs herança

- **Herança (extends + blocks)** Define a estrutura geral da página (layout base).
- **Include** Insere trechos de HTML prontos (componentes simples e reutilizáveis).
- **Macros** Funcionam como funções: recebem parâmetros e geram HTML dinamicamente.

Regra prática:

- Use extends para layout;
- Use include para componentes fixos;

- Use macro quando precisar de parametrização.

Exemplo adicional: banner condicional

Código: main/partials/promocao.html

```
1 <div class="promo" style="
2     background: linear-gradient(135deg, #ff6b6b 0%, #ff8e53 100%);
3     color: white;
4     padding: 20px 30px;
5     margin: 20px auto;
6     max-width: 900px;
7     border-radius: 12px;
8     box-shadow: 0 8px 20px rgba(255, 107, 107, 0.3);
9     text-align: center;
10    font-size: 1.6em;
11    font-weight: bold;
12    border: 3px solid #fff;
13    position: relative;
14    overflow: hidden;
15 ">
16    <!-- Efeito de brilho sutil (opcional, mas dá um toque premium) -->
17    <div style="
18        position: absolute;
19        top: -50%;
20        left: -50%;
21        width: 200%;
22        height: 200%;
23        background: radial-gradient(circle, rgba(255,255,255,0.3) 0%, transparent
24            70%);
25        animation: shine 6s infinite linear;
26        pointer-events: none;
27    "></div>
28    <strong style="letter-spacing: 1px;">
29        Promocao do Dia:
30    </strong>
31    {{ promocao | upper }} <!-- ou sem filtro, dependendo do conteúdo -->
```

```

32
33     <!-- Ícone ou destaque extra (opcional) -->
34     <span style="font-size: 2em; margin-left: 15px;"></span>
35 </div>
36
37 <!-- Animação simples de brilho (adicione no <head> ou em CSS global se preferir)
38 -->
39 <style>
40     @keyframes shine {
41         0% { transform: translateX(-100%) translateY(-100%); }
42         50% { transform: translateX(100%) translateY(100%); }
43         100% { transform: translateX(-100%) translateY(-100%); }
44     }
45 </style>

```

Uso no template index.html:

Código: main/index.html (inclusão)

```

1 {% if promocao %}
2     {% include 'main/partials/promocao.html' %}
3 {% endif %}

```

Precisamos agora registrar a promoção na rota "/":

Código: delivery/views/main.py (trecho)

```

1 @bp_main.route('/')
2 @bp_main.route('/index')
3 def index():
4     current_app.logger.debug("Renderizando template index.html")
5     user = {'username': 'wanderson'}
6     return render_template('main/index.html',
7                             user=user,
8                             promocao="Pizza em dobro nas quartas!")

```

Note que a diretiva `include` permite dividir templates em partes menores e reutilizáveis, melhorando a organização e a legibilidade do código. Combinada com herança e macros, ela permite construir interfaces complexas de forma modular, seguindo boas práticas de

engenharia de software.

NOTA

Na prática, aplicações reais utilizam uma estrutura baseada em layouts (herança), componentes (include) e elementos dinâmicos parametrizados (macros).

Na próxima etapa, vamos testar a renderização dos templates diretamente no ambiente interativo do Flask.

Testando Templates no Flask Shell

Durante o desenvolvimento, nem sempre é necessário iniciar o servidor web para testar templates. O Flask oferece um ambiente interativo (shell) que permite executar trechos de código isoladamente, o que é extremamente útil para depuração e experimentação.

Inicie o shell com:

```
flask shell
```

Esse comando abre um ambiente Python já configurado com a aplicação Flask carregada.

Renderização básica

Dentro do shell:

```
1 from flask import current_app, render_template
2
3 with current_app.app_context():
4     html = render_template(
5         "main/index.html",
6         user={"username": "Wanderson"}
7     )
8     print(html)
```

Esse código:

- carrega o template `main/index.html`;
- injeta variáveis no contexto (`user`);
- executa o Jinja2;
- retorna o HTML final renderizado.

Testando loops e listas

Podemos testar templates mais complexos, como o carrinho:

```
1 with current_app.app_context():
2     carrinho = {
3         "itens": [
4             {"nome": "Pizza", "preco": 49.9, "quantidade": 1},
5             {"nome": "Refrigerante", "preco": 8.5, "quantidade": 2}
6         ]
7     }
8
9     html = render_template(
10         "main/carrinho.html",
11         carrinho=carrinho,
12         total=66.9
13     )
14
15 print(html)
```

Isso permite validar rapidamente:

- loops (`for`);
- condicionais (`if`);
- macros e includes;
- formatação de valores.

Testando cenários de borda

Uma prática importante em ambientes profissionais é testar cenários alternativos:

```

1 with current_app.app_context():
2     carrinho = {"itens": []}
3
4     html = render_template(
5         "main/carrinho.html",
6         carrinho=carrinho
7     )
8
9     print(html)

```

Esse teste permite verificar se:

- o bloco `{% else %}` do loop funciona corretamente;
- mensagens para listas vazias são exibidas;
- o template não quebra com dados incompletos.

Por que o `app_context` é necessário?

O bloco:

```

1 with current_app.app_context():

```

é obrigatório porque muitos recursos do Flask dependem de um **contexto de aplicação ativo**. Sem ele, o Flask não sabe qual aplicação está sendo usada.

O contexto fornece acesso a:

- `current_app` (instância da aplicação);
- `config` (configurações);
- `g` (variáveis globais da requisição);
- sistema de templates (Jinja2);

- `url_for` e outros utilitários.

Sem esse contexto, você receberá erros como:

```
RuntimeError: Working outside of application context
```

Uso prático no desenvolvimento

O Flask shell é amplamente utilizado em ambientes profissionais para:

- testar templates sem subir o servidor;
- validar dados antes de integrar com banco de dados;
- depurar erros de renderização;
- experimentar rapidamente alterações na interface;
- inspecionar o comportamento de filtros, loops e macros.

NOTA

O Flask shell é uma ferramenta poderosa para testes rápidos e depuração durante o desenvolvimento. Ele permite validar templates de forma isolada, reduzindo o ciclo de feedback e facilitando a identificação de erros antes mesmo da execução completa da aplicação.

Na próxima seção, avançaremos para uma abordagem ainda mais poderosa: o uso do **Flask-DebugToolbar**, que permite inspecionar visualmente o contexto da aplicação, tempo de execução, queries SQL e muito mais diretamente no navegador.

Flask-DebugToolbar — Depuração Visual Profissional

A extensão `Flask-DebugToolbar` é uma das ferramentas mais importantes no desenvolvimento com Flask. Inspirada no Django Debug Toolbar, ela adiciona um painel

interativo no navegador que permite inspecionar, em tempo real, o comportamento interno da aplicação.

Com a toolbar ativa, cada requisição HTTP passa a exibir informações detalhadas que auxiliam no diagnóstico de problemas e na compreensão do fluxo da aplicação.

Entre os principais recursos, destacam-se:

- Visualização dos templates renderizados e das variáveis disponíveis no contexto;
- inspeção de tempo de execução da requisição;
- análise de logs gerados pela aplicação;
- acesso às configurações ativas do Flask;
- inspeção de queries SQL (quando integrado com banco de dados);
- editor de templates em tempo real (modo DEBUG)².

NOTA

Importante

A Flask-DebugToolbar já está declarada como dependência do projeto (via `pyproject.toml`), não sendo necessário instalá-la manualmente.

Seguindo o padrão industrial adotado ao longo do projeto, a extensão deve ser isolada em um módulo próprio antes de ser instanciada. Fazemos isso através da edição do seguinte arquivo.

Código: `ext/debugtoolbar/__init__.py`

```
1 from flask_debugtoolbar import DebugToolbarExtension
2
3 toolbar = DebugToolbarExtension()
4
5 def init_app(app):
```

²Recentemente o Flask-DebugToolbar passou por uma atualização e é possível que essa edição não esteja disponível nas versões atuais.

```

6     """Inicializa a Flask-DebugToolbar na aplicacao."""
7     toolbar.init_app(app)

```

A DebugToolbar depende de uma chave secreta para funcionar corretamente. Sendo assim, precisamos criar e ajustar um arquivo de configuração para o projeto.

Código: ext/config/__init__.py

```

1 def init_app(app):
2     app.config["SECRET_KEY"] = "dev-key" # Nunca usar em producao
3     app.config["DEBUG"] = True
4
5     if app.debug:
6         app.config["DEBUG_TB_TEMPLATE_EDITOR_ENABLED"] = True
7         app.config["DEBUG_TB_INTERCEPT_REDIRECTS"] = False

```

Em seguida, registramos a extensão e o arquivo de configuração no *pipeline* da factory:

Código: app.py

```

1 def create_app():
2     app = Flask(__name__)
3
4     # Configuracoes da aplicacao
5     from delivery.ext.config import init_app as init_config
6     init_config(app)
7
8     # Inicializacao da DebugToolbar (apenas em desenvolvimento)
9     from delivery.ext.debugtoolbar import init_app as init_toolbar
10    init_toolbar(app)
11
12    # Registro de blueprints
13    from delivery.views import init_app as init_views
14    init_views(app)
15
16    return app

```

 Aviso

Atenção de segurança: a DebugToolbar deve ser utilizada exclusivamente em ambiente de desenvolvimento. Ela expõe informações internas da aplicação que não devem ser acessíveis em produção.

Como visualizar a toolbar

Após iniciar o servidor:

```
flask run
```

Acesse qualquer rota da aplicação (por exemplo, <http://localhost:5000/>).

A toolbar aparecerá automaticamente no canto da tela (geralmente à direita), exibindo um painel expansível com diversas abas de inspeção.

Explorando os painéis principais

- **Templates** Mostra qual template foi renderizado e todas as variáveis disponíveis no contexto. Esse painel é extremamente útil para depurar problemas com Jinja2.
- **Config** Exibe todas as configurações ativas da aplicação Flask.
- **Logs** Mostra mensagens registradas via `current_app.logger`.
- **Time** Informa o tempo total de execução da requisição.
- **Profiler** Permite análise detalhada da execução (deve ser ativado manualmente).
- **SQLAlchemy** Exibe queries executadas (quando integarmos banco de dados nas próximas unidades).

Conexão com templates e contexto

Um dos usos mais importantes da DebugToolbar é a inspeção do contexto de templates.

Por exemplo, ao acessar a rota:

```
1 @bp_main.route('/')
2 def index():
3     user = {'username': 'Wanderson'}
4     return render_template('main/index.html', user=user)
```

No painel **Templates**, será possível visualizar:

- todas as variáveis disponíveis (como `user`);
- o valor de cada variável;
- a hierarquia de templates (base + filhos);
- includes e macros utilizados.

Isso elimina a necessidade de múltiplos `print()` e facilita a identificação de erros de renderização.

NOTA

Com a introdução da Flask-DebugToolbar, fechamos o ciclo fundamental do desenvolvimento web com Flask: criação da aplicação com Application Factory, organização modular com Blueprints, separação de responsabilidades com Templates, construção de interfaces dinâmicas com Jinja2 e, finalmente, inspeção e depuração profissional da aplicação.

A partir deste ponto, o projeto está preparado para evoluir de forma consistente, incorporando banco de dados, autenticação, formulários e outras funcionalidades típicas de aplicações reais.

Na próxima unidade, avançaremos na construção do sistema, integrando novas camadas à arquitetura e aprofundando o uso de extensões do ecossistema Flask.

Lista de Exercícios

5.1 No padrão *Application Factory*, por que não é recomendado utilizar `@app.route()` diretamente em módulos importados antes da criação da aplicação?

- (A) Porque o Flask não permite múltiplas rotas no mesmo arquivo
- (B) Porque o decorador `@app.route()` só funciona em ambiente de produção
- (C) Porque a instância `app` ainda não existe no momento da importação, causando erro de atributo ✓
- (D) Porque rotas devem sempre ser declaradas dentro do `__main__`

Justificativa: No padrão *Factory*, `app` só é criado em tempo de execução. Tentar usar `@app.route()` antes disso gera erro, pois `app` ainda não foi instanciado.

5.2 Considere o código abaixo:

```
1 bp = Blueprint("main", __name__)
2
3 @bp.route("/")
4 def index():
5     return "OK"
```

Para que essa rota funcione corretamente, é necessário:

- (A) Importar o blueprint no template
- (B) Registrar o blueprint com `app.register_blueprint(bp)` ✓
- (C) Usar `@app.route()` no lugar de `@bp.route()`
- (D) Criar uma classe base para o blueprint

Justificativa: Blueprints são estruturas “dormentes” e precisam ser registrados explicitamente na aplicação para que suas rotas sejam ativadas.

5.3 Analise o código:

```
1 return render_template("main/index.html", user=user)
```

No template:

```
1 {{ user.username }}
```

Qual afirmação é correta?

- (A) user não pode ser acessado no template
- (B) O template não suporta acesso por ponto
- (C) É obrigatório usar user['username']
- (D) user.username acessa chave de dicionário ou atributo ✓

Justificativa: O Jinja2 permite acesso tanto a atributos quanto a chaves de dicionário via notação por ponto.

5.4 Considere o template:

```
1 {% for item in itens %}  
2     {{ item }}  
3 {% else %}  
4     Lista vazia  
5 {% endfor %}
```

O bloco else será executado quando:

- (A) O loop não itera nenhuma vez ✓
- (B) Se houver mais de 10 itens
- (C) Apenas se ocorrer erro
- (D) Apenas em modo DEBUG

Justificativa: O Jinja2 executa o else quando o loop não itera nenhuma vez, o que é o caso de uma lista vazia.

5.5 Analise:

```
1 {{ preco | round(2) }}
```

Esse código:

- (A) Arredonda o valor para 2 casas decimais ✓
- (B) Converte string para inteiro
- (C) Remove casas decimais
- (D) Converte para string

Justificativa: O filtro `round(2)` arredonda o valor numérico para duas casas decimais.

5.6 Sobre o uso de `current_app` em blueprints, assinale a alternativa correta:

- (A) Substitui o Flask completamente
- (B) Só funciona fora de rotas
- (C) Permite acessar a aplicação ativa sem referência direta ao `app` ✓
- (D) É obrigatório em todas as funções

Justificativa: O `current_app` é um proxy que aponta para a instância ativa da aplicação.

5.7 Sobre o autoescape do Jinja2:

- (A) Ele criptografa dados
- (B) Ele remove scripts automaticamente
- (C) Ele escapa HTML para prevenir XSS ✓
- (D) Ele impede SQL Injection

Justificativa: Autoescape transforma caracteres HTML em entidades seguras, prevenindo XSS.

5.8 Qual o efeito do filtro `|safe`?

- (A) Permite renderizar HTML sem escape ✓
- (B) Executa código JavaScript
- (C) Converte string em JSON

(D) Cria variável global

Justificativa: O filtro `safe` desativa o autoescape para aquele conteúdo específico.

5.9 Considere:

```
1 {% extends "main/base.html" %}
```

Esse comando:

- (A) Importa funções Python
- (B) Define herança de template ✓
- (C) Cria variável global
- (D) Substitui o Jinja2

5.10 Sobre `{{ super() }}` em templates:

- (A) Só funciona em loops
- (B) Executa código Python
- (C) Remove conteúdo do bloco pai
- (D) Mantém conteúdo do bloco pai e adiciona conteúdo extra ✓

5.11 Considere:

```
1 {% include 'main/partials/navbar.html' %}
```

Esse comando:

- (A) Insere template parcial no ponto atual ✓
- (B) Substitui blueprint
- (C) Reinicia contexto
- (D) Cria variável

5.12 Sobre macros:

- (A) São equivalentes a classes Python

- (B) Funcionam como funções reutilizáveis de template ✓
- (C) Não aceitam parâmetros
- (D) Só funcionam em `base.html`

5.13 No Flask shell, por que usamos:

```
1 with current_app.app_context():
```

- (A) Para ativar debug
- (B) Para iniciar servidor
- (C) Para permitir acesso ao contexto da aplicação ✓
- (D) Para criar blueprint em tempo de execução da aplicação

5.14 Qual erro ocorre ao usar `render_template` sem contexto?

- (A) `KeyError`
- (B) `TypeError`
- (C) `SyntaxError`
- (D) `RuntimeError: Working outside of application context` ✓

5.15 Sobre `DebugToolbar`:

- (A) Permite inspecionar contexto e execução ✓
- (B) Remove erros automaticamente
- (C) Deve ser usada em produção
- (D) Não mostra templates

5.16 Em relação ao padrão em camadas:

- (A) Views devem acessar banco diretamente
- (B) Models representam dados ✓
- (C) Templates executam SQL

(D) Blueprints substituem models

5.17 Considere:

```
1 {{ lista | join(', ') }}
```

Esse código:

- (A) Junta elementos em string ✓
- (B) Remove elementos
- (C) Converte em JSON
- (D) Executa loop

5.18 Sobre separação de responsabilidades:

- (A) Models controlam rotas
- (B) Views devem gerar HTML manualmente
- (C) Templates devem cuidar da apresentação ✓
- (D) Templates devem conter lógica de negócio

5.19 Sobre `url_for` em templates:

- (A) Retorna HTML
- (B) Gera URLs dinamicamente ✓
- (C) Substitui blueprint
- (D) Cria rota automaticamente

5.20 Qual das opções representa melhor a evolução arquitetural apresentada no nosso estudo?

- (A) HTML inline → Flask sem rotas → SQL direto
- (B) Rotas simples → Blueprints → Templates → Arquitetura em camadas ✓
- (C) Factory → CSS → JS
- (D) Python → HTML → Docker

Justificativa: O conteúdo trabalhado apresentou a construção básica até uma arquitetura modular e escalável, alinhada ao padrão profissional.